# Statistical Data Value Chain

## Table of contents


Value Chains Introduction . . . . . 1
Purpose of this chapter . . . . . 1
Where to find what . . . . . 1
The chain at a glance . . . . . 2
The four process types . . . . . 3
How each value stream is described . . . . . 3
Identifiers used in this chapter . . . . . 4
Steady States of Data . . . . . 4
Applying the steady states pragmatically . . . . . 5
Principles to Apply . . . . . 5


## Value Chains Introduction

### Purpose of this chapter

This chapter describes how the statistical value chain works in practice. The Strategy chapter introduced the ten value streams, their stages, and the value they create. The Steady-State Architecture describes the results the chain hands over: the states, their quality gates, and their metadata. This chapter describes the work between those results: the stages, the processes that realize them, the roles that carry them out, and the events that connect them. Every value stream is described with the same structure, so that NSIs can compare them and map their own organization onto them. **[VC1]**

### Where to find what

| Question | Answered in |
|---|---|
| Which value streams exist, and what value do they create? | Strategy, *Value Streams* |
| What must data satisfy at each handover point? | Steady-State Architecture, per state |
| How is the work in each value stream organized? | This chapter |
| Which actors, roles, processes, and functions exist? | Business Architecture, reference content |
| How mature is an NSI, and how does it apply a value stream? | SAF Toolkit |

The division of work between states and value streams is deliberate. A state describes the **result**: what the data looks like, which gate it passed, and how it is protected. A value stream describes the **work**: who does what, in which order, and with which controls. Gate criteria and minimum metadata sets are therefore not repeated in this chapter. Each value stream refers to the state it produces.

**Key concepts — if you are new to architecture documents** This documentation uses a handful of architecture terms. Here is what they mean in plain language:

- **Value stream** — a sequence of steps that together create a statistical output. Think of it as a production line: VS-01 to VS-06 each add a specific kind of value to the data.
- **Value stream stage vs. business process** — the stage (`VS-0x.y`) says *what value* is added; the business process describes *how* your office actually does it. The numbering maps one-to-one.[^archimate]
- **Steady state** — a fixed handover point: a versioned, quality-assured dataset that one stream produces and the next one consumes. The framework defines five numbered steady states, #1 to #5; data still at the external provider (#0) and data in users' hands (#6) sit outside the NSI's control and are boundary positions, not numbered states.
- **The four process types** — every domain is described through the same four groups: **Design** (decide how it should work), **Implementation** (build and test it), **Execution** (run it each production cycle), **Management** (check quality and feed improvements back).
- **Business service / object / event** — what a domain offers to others / the data artefacts it works on / the signals streams send each other (e.g. *"Raw Data Registered"* tells the next stream new data is ready).
- **NSI** — National Statistical Office.
- **GSBPM** — the *Generic Statistical Business Process Model*: the standard, internationally agreed description of the phases of statistical production (specify needs, design, build, collect, process, analyse, disseminate, evaluate).
- **GAMSO** — the *Generic Activity Model for Statistical Organizations*: the companion standard describing management and support activities (strategy, capability development, corporate support).
- **SAF-P01 ... SAF-P11** — the eleven SAF architecture **p**rinciples, defined in the SAF Framework document and referenced throughout.

### The chain at a glance

Six primary value streams form the production chain. Each one consumes a registered state version and produces the next. They are loosely coupled (SAF-P07): a value stream needs no knowledge of the internal work of its neighbors, only of the state it receives and the state it delivers. The handover is triggered by a registration event. Four supporting value streams run continuously across the whole chain. They set the rules, develop new methods, provide the platforms, and keep the chain connected to its users and data providers.

| VS | Value stream | Type | State in → out | Event out |
|---|---|---|---|---|
| VS-01 | Data Acquisition and Ingestion | Primary | #0 → #1 | *Raw Data Registered* |
| VS-02 | Data Standardization and Integration | Primary | #1 → #2 | *Standardized Data Registered* |
| VS-03 | Data Editing and Enrichment | Primary | #2 → #3 | *Processed Data Registered* |
| VS-04 | Estimation, Analysis and Statistical Modeling | Primary | #3 → #4 (analytical) | *Output Data Registered* |
| VS-05 | Statistical Product Composition | Primary | #4 (analytical) → #4 (approved) | *Statistical Product Registered* |
| VS-06 | Dissemination and Service Delivery | Primary | #4 (approved) → #5 | *Statistics Released* |
| VS-07 | Governance and Compliance | Supporting | Across the chain | n/a |
| VS-08 | Innovation and Methodological Development | Supporting | Across the chain | n/a |
| VS-09 | Infrastructure and Platform Enablement | Supporting | Across the chain | n/a |

| VS | Value stream | Type | State in → out | Event out |
|---|---|---|---|---|
| VS-10 | Customer and Stakeholder Engagement | Supporting | Across the chain; feedback from #5 and #6 | n/a |

Not every statistical product passes through every primary value stream. A macro statistic, for example, often starts from statistics another domain has already registered. Such a product enters the chain downstream and its lineage points to the state it consumed (see *Applying the steady states pragmatically*). Data that comes from outside the NSI always enters through VS-01.

### The four process types

Every business domain, and therefore every primary value stream, runs the same four process types. **Design** decides how the work is done and delivers metadata: designs, rules, and specifications. **Implementation** builds and tests the solution and delivers the execution process. **Execution** runs the work in each production cycle and delivers the statistical data; the stages of a value stream describe this execution. **Management** monitors the work, assesses quality, and feeds improvements back to design. Design and implementation only run when something changes, execution runs every cycle, and management runs continuously.

The supporting value streams do not follow a production cycle. They run the same pattern as a continuous cycle of plan, build, operate, and improve.

| Process type | Provides | GSBPM phases | Equivalent in supporting streams |
|---|---|---|---|
| Design | Metadata: designs, rules, specifications | 1 Specify Needs, 2 Design | Plan |
| Implementation | The execution process: instruments, components, configured workflows | 3 Build | Build |
| Execution | Statistical data, registered in a state | 4 Collect, 5 Process, 6 Analyze, 7 Disseminate | Operate |
| Management | Quality reports and improvements | 8 Evaluate; overarching quality and metadata management | Improve |

### How each value stream is described

Each primary value stream is described with the same ten attributes. The supporting value streams use the same attributes, with two differences: attribute 2 describes the relation to the chain instead of a state transition, and attribute 4 follows the cycle of plan, build, operate, and improve.

| # | Attribute | Question it answers |
|---|---|---|
| 1 | Purpose and value created | Why does this value stream exist, and what value does it add? |
| 2 | Position and state transition | Which state does it consume, and which does it produce? |
| 3 | Stages | Which stages create the value, and what happens in each? |
| 4 | The four process types | How is the value stream designed, built, and managed? |
| 5 | Actors and roles | Who is involved, and with which responsibility? |

| # | Attribute | Question it answers |
|---|---|---|
| 6 | Capabilities, processes, and functions | Which SAF capabilities, processes, and functions realize the stages? |
| 7 | Principles | Which architecture principles weigh most, and how are they applied? |
| 8 | Interfaces, services, and events | What does the value stream offer and exchange, and what triggers it? |
| 9 | Key performance indicators | How is the performance of the value stream measured? |
| 10 | Examples | What does the value stream look like in practice? |

Each value stream has three ArchiMate views: (a) the value stream with its stages and the capabilities that serve them, (b) the stages and the processes and functions that realize them, and (c) the state handover and its events. Maturity checklists and detailed end-to-end scenarios for each value stream are part of the SAF Toolkit. [**VC2**]

#### Identifiers used in this chapter

| Identifier | Refers to | Defined in |
|---|---|---|
| VS-01 to VS-10, VS-01.1, … | Value streams and their stages | Strategy, *Value Streams* |
| 1.2.1, 2.1.2, … | Capabilities | Strategy, *Capability Model* |
| ACT, ROL, COL | Business actors, roles, and collaborations | Business Architecture |
| PRD, OVA, DLM, MDS, DSP | Business processes | Business Architecture, *Business Processes* |
| FUN | Business functions | Business Architecture, *Business Functions* |
| #0 to #6 | Steady states and boundary positions | Steady-State Architecture |
| SAF-P01 to SAF-P11 | Architecture principles | Motivation, *Architecture Principles* |

### Steady States of Data

The streams work independently of each other (in architecture terms: they are **loosely coupled**). They exchange data only at defined **steady states** — points where a high-quality, versioned dataset is handed over. In the usual case each primary stream consumes one steady state and produces the next, so a stream needs no knowledge of its neighbors' internals (SAF-P07). The handover works through events: when a stream registers a finished dataset, a notification tells the next stream that new data is ready. Not every process has to pass through every state; see Applying the steady states pragmatically.

Of the five, **processed data (#3) is the only steady state mandatory for all production** — it is the microdata on which everything downstream is verifiably built. **Raw Data (#1)** is also mandatory when a product collects its own data, but not for a macro statistic that starts entirely from another domain's data. **Standardized data (#2)** is always optional, and **Statistics (#4)** may share its artefact with **Released data (#5)** when the two are identical.

### Applying the steady states pragmatically

A steady state is an agreement about data: the quality it has reached, who owns it, how it is identified and what metadata comes with it. It does not require a separate physical copy at every state, and it does not require every process to pass through all six. An office may depart from the usual sequence in the three cases below, provided it records the choice (which state was shared or skipped, and why) in the catalogue entry. In other words, the rule is "comply or explain".

1. **Two states can share one artefact.** If nothing happens between State #4 and State #5 (no transformation and no further disclosure control), the #4 artefact does not have to be registered again as a new dataset; it is marked as released instead. The release steps still take place: release authorisation is obtained and the final disclosure check is carried out, and both are recorded even when the check changes nothing.
2. **A process can start downstream.** Not every production process begins at VS-01. Macro statistics, for example, are often built on Processed Data (#3) or Statistics (#4) that another team has already registered, and State #4 above can already be "shared internally for use as input data in other statistical processes". Such a process starts from the registered artefact and its lineage points to it; there is no need to create placeholder #1 or #2 artefacts. Data that comes from outside the office still enters through VS-01 and State #1.
3. **Lineage is about traceability, not duplication.** SAF-P09 requires that every value can be traced back to where it came from. When a stream uses an artefact without changing any values, a logged process event that refers to the existing artefact version meets this requirement, and no identical copy needs to be registered. As soon as a value changes, a new version is registered.

One limit applies to all three cases. States must not be merged where this would hide a change of values (for example, standardisation at #2 and editing at #3 carried out as a single undocumented step) or a disclosure-control step (for example, #4 and #5 when additional disclosure control is applied before release).

## Principles to Apply

The SAF defines eleven architecture principles, labeled SAF-P01 to SAF-P11. The six **primary** streams actively apply six of them (marked  below); the four **supporting** streams foreground the other five (marked †). The table shows each principle with practical guidance on how to apply it; VS-07 in particular **enforces** the data principles (P09/P10/P11) across the whole chain.

Table 1: SAF Principles

| Principle | Statement | How to apply across the value chain |
|---|---|---|
| **SAF-P01** | Actively participate in open-source communities to maintain collective control over statistical processes | Build statistical capabilities on shared, open tooling; contribute back, so NSIs keep collective control of critical methods rather than depending on vendors. |
| **SAF-P07** | Implement loosely coupled processes and systems | Exchange data only at steady-state interfaces; each stream consumes one state and produces the next without knowing the others' internals. |
| **SAF-P08** | Data is shared property | Once an artefact is registered at a steady state, make it available to all authorized consumers; avoid duplicating collection, processing, or analysis. |
| **SAF-P09** | No data without metadata and classification | Every artefact carries provenance (origin), methodology, accuracy, classification, and lineage (the trace back to its inputs) — captured at the point each value is created or changed. |
| **SAF-P10** | Security By Design | Embed access control, integrity verification, and tamper-evident audit trails in every pipeline from the start. |
| **SAF-P11** | Privacy By Design | Remove/pseudonymize identifiers during standardization; assess disclosure risk on enrichment and apply statistical disclosure control before release. |

| Principle | Statement | How to apply across the value chain |
|---|---|---|
| **SAF-P02** † | Active support | Operate and support capabilities as living services, not one-off builds (VS-09 platforms, VS-10 services). |
| **SAF-P03** † | Active Life Cycle management | Version and govern rules, methods, models, and products across their life cycle; retire what is obsolete (VS-07, VS-08). |
| **SAF-P04** † | Maximize the benefits for the organization | Prefer solutions that serve the whole organization and real user value over local optimizations (VS-10, VS-07). |
| **SAF-P05** † | IT and technology is everyone's business | Treat methodology, data, and technology as a shared responsibility across business and IT (VS-08, VS-09). |
| **SAF-P06** † | Business Continuity | Keep the chain running reliably and able to recover from disruption (VS-09). |

= actively applied by the six primary value streams. † = foregrounded by the four supporting value streams (VS-07–VS-10).

**Metadata everywhere (SAF-P09):** value is only added once its context is captured — what was done, by which rule/method, with what accuracy, and traceable to its inputs.

**Confidentiality progressively applied (SAF-P11):** identifiers are removed/pseudonymized at standardization (#2) at the latest, or earlier where that suits the process; disclosure risk is assessed on enrichment (#3), disclosure control is applied to outputs (#4), and a final disclosure check gates release (#5).